Pocket FT8 Transceiver Revisited

Jim Conrad, KQ7B
132 Mill Creek MDW, Grangeville, ID 83530 USA

The Pocket FT8 Transceiver Revisited is a self-contained, portable software defined QRPP transceiver.

Introduction and Features

Charles Hill's original Pocket FT8 project[1] inspired several fully self-contained FT8 transceiver derivatives[2][3] in which the computer *is* the radio. This derivative revisits the original concept with simplified hardware and enhanced firmware while retaining the pocket-size, single-band, QRPP, HF transceiver concept for outdoor adventuring. Features include:

- Open source hardware and firmware
- Self-contained and light-weight
- Compact, 4-layer PCB (4.0X2.8")
- FSK FT8 modulator/demodulator
- 275+ mW RF output (QRPP)
- RF filter slot
- Low jitter TCXO
- GPS disciplined date, time and location (grid square)
- 480x320 pixel TFT calibrated touchscreen
- Waterfall traffic display
- Standard FT8 QSO sequencer, "RoboOp"
- micro-SD card accessible via USB
- Automatic QSO logging to an ADIF file on SD
- JSON station configuration file on SD
- Operation from a 5V USB power bank

The FT8 protocol[4], designed for weak signal communication, excels in portable, battery-operated, QRPP operation. While traditional setups consist of a computer, transceiver, and, in some cases, an audio interface. Pocket FT8 integrates these functions in a single, self-contained package. Use of FT8 reduces the transceiver's power requirements and weight while also eliminating the telegraph key, microphone, and headset accessories accompanying SSB/CW transceivers. Consolidating the radio with the FT8 encoder/decoder in an embedded computer eliminates the need to carry a computer into the field. Further integration of a GPS receiver offers UTC date and time for ADIF logging as well as the Maidenhead grid square and synchonization with FT8 time-slots.

FT8 QSOs are terse, contest-like exchanges conveying sequenced, structured messages with minimal operator intervention after contact establishment. Because an FT8 message

requires an extremely quick response, WSJT-X[4] and related FT8 applications automate the message exchange between stations as human response times are woefully inadequate. Pocket FT8 Revisited includes a QSO sequencer, affectionately known as, RoboOp, implementing WSJT-X style message sequencing for Tx1 through Tx6. Akin to WSJT-X, RoboOp retransmits the previous message if it doesn't receive a valid response. A configurable QSO timer guards against endless retransmissions.

The Teensy micro SD card slot holds both the operator-supplied configuration file, CONFIG.JSON, and the ADIF log file produced during station operation. The executing firmware supports the Media Transfer Protocol[5] (MTP) enabling an optional host computer to access the Teensy SD card via USB without removing the media. MTP is often the preferred way to read the log file when the SD card is not accessible through the transceiver's enclosure.

Transceiver Construction

TBD

Building the Firmware

The firmware was developed on MacOS but should build on Windows or linux. In all cases, use Visual Studio Code and PlatformIO to build and install the firmware. Install the required development tools for your computer's operating system and project source code:

- Visual Studio Code
- C/C++ 23 compiler/linker
- `PocketFT8XcvrFW` source code

You will need some Visual Studio Code extensions for C/C++ embedded systems development. From within Visual Studio Code, install:

- PlatformiO IDE
- C/C++ DevTools
- C/C++ Extension Pack
- C/C++Themes
- CMake Tools

You will likely need to edit the `PocketFT8XcvrFW/platform.io` project configuration for your development environment, especially parameters describing which USB port connects to Teensy, and the `lib_extra_dirs` pathnames.

You can test your development environment by having PlatformIO build `test_01`, an implementation of blinky, with a pio command, `pio test -f hw/test_01`.

Installing and Debugging the Firmware

Connect your development computer to Teensy's USB port. With all `platformio.ini` references to ports commented-out, click on PlatformIO's "Upload" icon. If all goes well,

PlatformIO will automatically locate the correct USB port and install the firmware. Upon occasion, PlatformIO may not dependably locate the correct port which you may resolve by manually configuring that port in `platformio.ini`.

Teensy 4.1 does not include a debugging port such as JTAG. Consequently, all debugging to date has been achieved using debugging messages written to Teensy's USB serial port. The `DEBUG.h` and `NODEBUG.h` contain C macros (e.g. DPRINTF) facilitating use of debugging messages. Including `DEBUG.h` in a source file enables that file's debugging messages, while including `NODEBUG.h` disables them.

Transceiver Configuration

You must use a computer to create a configuration file, `CONFIG.JSON`, on a micro SD card and install the card in the Teensy slot as the radio can not boot-up without it. After the radio is operational, all the files on the SD card are accessible via USB through MTP. Sample JSON configuration files appear in Figure 3 and in the github project's Examples folder.

Only two config parameter entries, the station's callsign and operating frequency, are essential; all remaining parameters illustrated in Figure 3 may be omitted. The callsign must conform to the WSJT-X requirements. The operating frequency must be provided in kHz (without a decimal fraction) and that frequency must be compatible with the low-pass filter installed in the filter slot, FL1.

The operator's personal name is optional and is included in log file entries when provided.

The optional M0, M1 and M2 entries program three possible "canned" 13-character FT8 "free text" messages.

When GPS is not reliable, the station's 4-character maidenhead grid square locator may be configured manually.

The Si4735 automatic volume control feature may be optionally configured off (`false`). The default is on (`true`).

To prevent endless retransmissions, RoboOp maintains a timer and abandons a QSO after the `qsoTimeout` period (in seconds) expires.

By default, RoboOp tracks recently worked stations and avoids contacting duplicates; the `enableDuplicates` parameter override this prohibition if set to `true`.

Successful QSOs, those exchanging callsigns and signal reports, are logged to an ADIF file whose name may be optionally configured to override the default, `LOGFILE.ADIF`. This is useful, for example, if you'd like to maintain different log files for different dates or QTHs. In all cases, the log file is written to the Teensy micro SD card.

If your station is located at a SOTA site, you may wish in configure a SOTA Reference Number to be included in the log entries.

Connections

The only truly required connections to the radio are power, antenna and the micro SD card containing the CONFIG.JSON file. Five volt power must be supplied through the Teensy 4.1's USB Micro-B port. Commonly available USB power banks are satisfactory. The HF

antenna must be connected to the transceiver's SMA port. The micro SD card must be inserted in the Teensy slot, not the unsupported Adafruit display slot.

The use of a connected GPS significantly improves the transceiver's usability and is strongly recommended. The GPS connection is made with a 5-pin ribbon cable to J3 for received data (`GPSRX`), transmitted data (`GPSTX`), Pulse Per Second (`PPS`), power (`+5V`) and ground (`GND`) as illustrated in Figure 1. A GPS fix automatically configures the station's maidenhead grid square locator as well as the UTC date and time. Lacking the GPS, the radio uses the manually configured locator and restores the current date and time from Teensy's battery-backed clock. Without the GPS, the station's timezone is unknown and QSOs may not be logged using UTC time as required by ADIF. Without the GPS, the transceiver's knowledge of the FT8 15-second time slot boundaries drifts with the battery-backed clock. Because FT8 requires transmissions to begin at accurately known time slots, communication is frustrated or impossible with timing errors. When a GPS fix has never been achieved, Teensy's battery-back clock is using date and time values obtained by the boot loader; these are unlikely to be particularly accurate nor UTC. Use the GPS whenever possible.

There is no microphone, key or headphone connection. The computer *is* the radio, and only digital modulation is supported.

User Interface

The transceiver's user interface consists of a waterfall display of activity within the receiver's passband, a station status display, a display of decoded messages, a transcript of the current QSO's traffic (if any), and a set of menu buttons. Brighter colors in the waterfall indicate stronger signals. The status display indicates the station's callsign, maidenhead grid square locator, date, time, etc. Values deemed certain appear in green text, while unvalidated content appear in yellow.

FT8's AFSK legacy specifies the operating frequency as the carrier frequency (in kHz) and an audio offset (in Hz); the actual unmodulated operating frequency is their sum. The transceiver's carrier frequency must be configured in CONFIG.SYS. The default offset is 1000 Hz and is changed by touching the waterfall display.

The transceiver boots up when 5V power is applied to the Teensy's USB port. The status display indicates the station's callsign and operating frequency as obtained from CONFIG.JSON. Until a GPS fix is obtained, the status display presents the battery-backed date, time and the grid square in yellow. Following a GPS fix, the UTC date and time, and the maidenhead grid square, appear in green text. During initialization, the transceiver reads the content of the log file, if any, and notes the callsigns of upto 1000 previously worked stations. By default, RoboOp will not rework those stations.

Following initialization, the transceiver begins displaying decoded FT8 messages from other stations. Received CQ messages are highlighted to gain your attention. The limited display size requires the signal strength of the decoded messages to appear as simple S values (e.g. S1, S2, ... S9) rather than dB as in WSJT-X. When the band is quiet, you may be able to contact almost any heard station; in a noisy band, only the stronger stations may be able to decode your QRPP transmissions. Touch a decoded station's message (e.g. CQ) to initiate a QSO with that station. Alternatively, you may use the CQ menu button to begin transmitting your own CQ messages. RoboOp (the firmware sequencing a WSJT-X-style QSO) will begin transmitting at the next appropriate FT8 time slots, avoiding those

time slotss when a called station will likely transmit. If you do not receive an appropriate response, RoboOp repeats your transmission until the configured timeout period expires, or you touch the ABort button. In all cases, RoboOp attempts to avoid transmitting in the same time slots as a specified remote is expected to transmit. In crowded band conditions, you may wish to change the offset frequency (by touching the waterfall) to select a quiet location.

Hardware Overview

The Pocket FT8 transceiver (Figure 1) is designed around a Teensy 4.1, 600 mHz, 32-bit ARM Cortex-M7 MCU with hardware floating point, USB, SPI, I2C, 1 mB of RAM, and two ADCs.

When transmitting, the MCU reprograms the Si5351 clock generator's output for each of the eight FT8 Frequency Shift Keying (FSK) tones, a direct FSK scheme, unlike the Audio FSK (AFSK) widely used with a SSB transmitters' microphone jack. Frequency stability is achieved by locking the Si5351 to a low jitter Temperature Controlled Crystal Oscillator (TCXO) reference. The Si5351's output is buffered by one element of a SN74ACT244 octal bus driver whose remaining seven buffers function in parallel as the transmitter's QRPP power amplifier (PA). A five-pole Chebyshev low-pass filter with 1 dB of ripple (Figure 2) peaking near the expected operating frequency provides harmonic rejection. The PA delivers 275 mW through the filter.

An Si4735 digital receiver implements the receiver's RF section and delivers FT8 audio tones to the Teensy audio pipeline via ADC2. While the Si4735 lacks the performance of alternative designs, its capabilities are on par with the QRPP transmitter: on-the-air experience found the receiver decodes more signals than the 275 mW transmitter can work, especially in crowded band conditions. The development of an early prototype found the Si4735 performance degrades in the presence of noise from other I2C bus device traffic; the problem was resolved by isolating the Si4735 on a private I2C bus. The Si4735's phase-locked loop (PLL) locks to the Si5351 clock generator's CLK2 output.

An Adafruit 3.5" TFT 320x480 touchscreen with an HX8357D controller serves as the graphical user interface device. The display achieves satisfactory performance over an SPI bus from the MCU while the resistive touchscreen connects to the MCU's ADC1. The compact display is ideal for portable operation, but touchscreen accuracy improves with use of a stylus rather than your finger.

FL1 Design

FL1 mainly provides harmonic rejection for the transmitted signal. On 40 meters, the five pole Chebyshev design provides for 1 dB of ripple peaking inside the band.

During receive, the Chebyshev low-pass design offers only minimal rejection of out-of-band signals. While this has not proven to be a significant issue in the wilderness use-case, the receiver might benefit from a bandpass design in other applications.

Hardware Construction

T1

T1 is a broadband, bifilar wound transformer consisting of 10 turns of 26 AWG on an FT37-43 core.

FL1

The FL1 low-pass filter can be assembled and tested on a small daughter board. With some effort, it can probably be made pluggable to facilitate band changes.

Software

Software Design

Software Construction

Testing

Operation

Future

Acknowledgements

References

[1] C. Hill. (2021, September) Pocket-ft8. [Online]. Available: https://github.com/Rotron/Pocket-FT8

[2] B. AŞUROĞLU. (2025, July) Dx ft8 a multiband ft8 digital mode 5 bands or 7 bands tablet transceiver. [Online]. Available: https://github.com/WB2CBA/DX-FT8-FT8-MULTIBAND-TABLET-TRANSCEIVER

[3] ——. (2026, May). [Online]. Available: https://antrak.org.tr/blog/tinydx-my-quest-for-designing-the-smallest-digital-modes-hf-transceiver/

[4] S. F. K. B. S. G. J. T. (K1JT), "The ft4 and ft8 communication protocols," *QEX*, July/August 2020.

[5] [Online]. Available: https://github.com/PaulStoffregen/Teensyduino_Examples/tree/master/USB_Media_Transfer/Basic_SD_Card

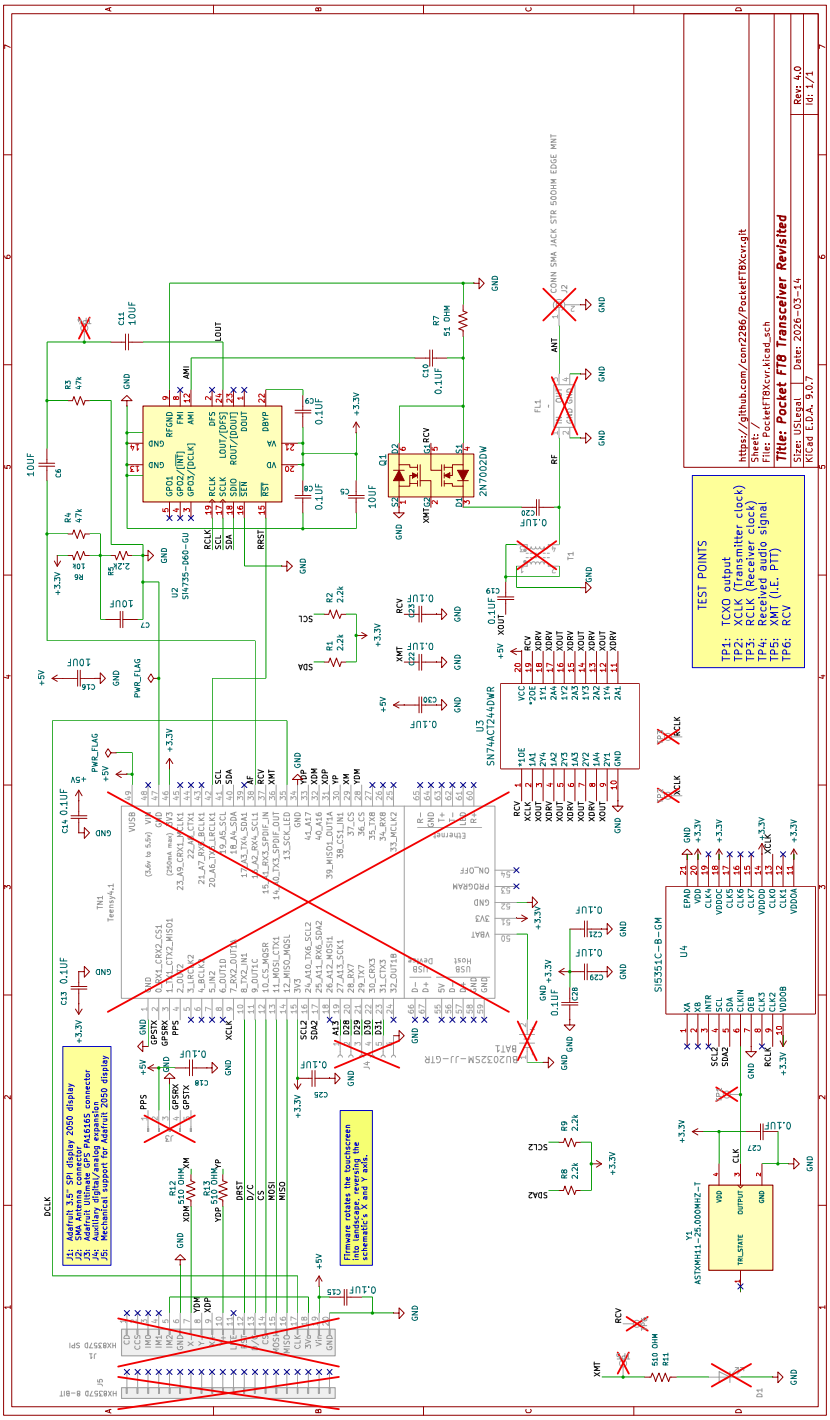

Figure 1: Schematic

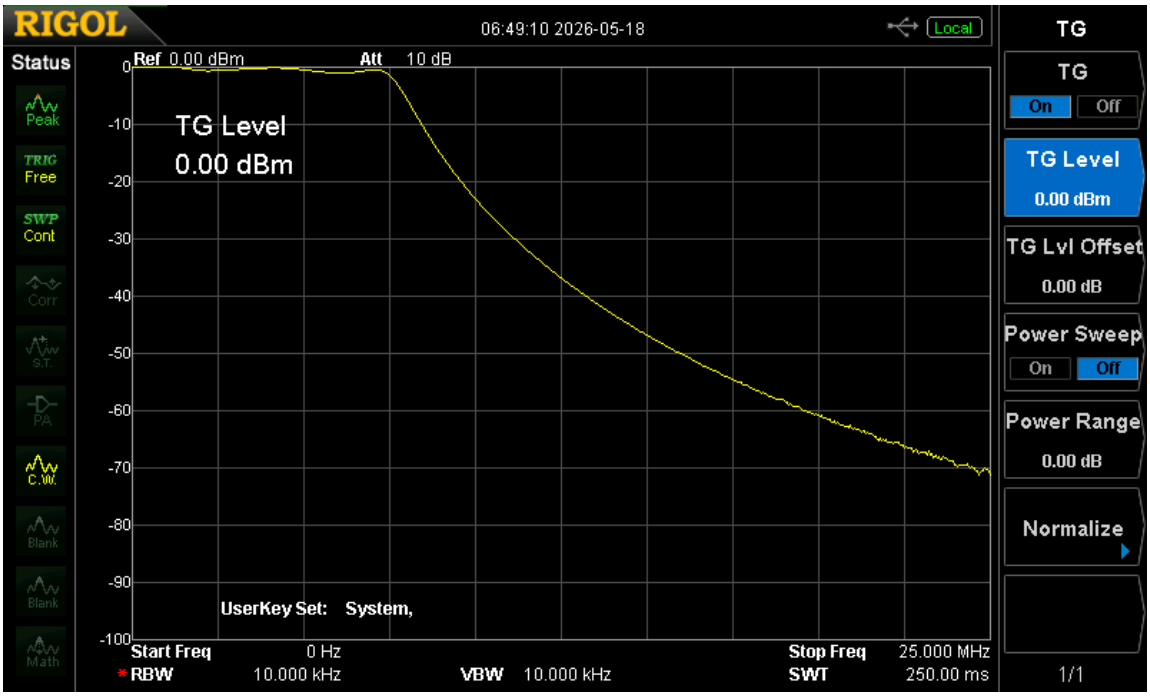

Figure 2: Chebyshev Low-Pass Filter Bode Plot

```
//-CONFIG.JSON--------------------------------------------------------------
{
   "callsign": "W1AW",              //REQUIRED:  Station callsign
   "frequency": 7074,               //REQUIRED:  Operating frequency in kHz
   "myName" : "Jim",                //OPTIONAL:  Operator's personal name
   "M0" : "CQ KQ7B/IDAHO",          //OPTIONAL:  13-Char Free Text Msg0
   "M1" : "IDAHO",                  //OPTIONAL:  13-Char Free Text Msg1
   "M2" : "QRT"                     //OPTIONAL:  13-Char Free Text Msg2
   "tcxoCorrection": -5,            //OPTIONAL:  TCXO freq correction in Hz
   "locator": "",                   //OPTIONAL:  4-char maidenhead grid
   "enableAVC": true,               //OPTIONAL:  0=disabled, 1=enabled
   "qsoTimeout": 180,               //OPTIONAL:  Seconds RoboOp will retry
   "enableDuplicates": false,       //OPTIONAL:  Enable duplicate contacts
   "logFilename" : "LOG0815.ADIF",  //OPTIONAL:  ADIF logfile name
   "my_sota_ref" : "W7I/IC-257",    //OPTIONAL:  SOTA Reference Number
}
```

Figure 3: CONFIG.JSON